

IntakeFlow - MVP Product Requirements Document (PRD)

Service Intake & Lead Pipeline Mini SaaS | Version 1.0 | Date: February 27, 2026

1. Product Overview

IntakeFlow is a lightweight web application that helps service businesses capture incoming requests (leads), qualify them with a small set of targeted questions, and track follow-ups through a fixed pipeline: **New** → **Contacted** → **Qualified** → **Won** → **Lost**. It is designed to be production-like while keeping the architecture intentionally simple: one backend API, one database, and one frontend.

2. Goals & Non-Goals

Goals

- Capture requests from customers with minimal friction (short form, 1–3 qualifying questions).
- Provide a staff dashboard to review requests, record follow-up activities (timeline), and move requests across pipeline stages.
- Provide actionable analytics (conversion rates, funnel, time-to-first-contact, outcomes).
- Keep architecture simple and deployable with Docker locally, then Kubernetes on AWS.
- Use only free/open-source tools and free tiers where applicable.

Non-Goals (MVP)

- No full authentication/authorization system in MVP (admin access via secret link).
- No payments, invoicing, or ecommerce features.
- No notifications (email/SMS/WhatsApp) in MVP; only manual follow-up logging.
- No drag-and-drop form builder or domain-specific custom implementations.

3. Target Users & Primary Flows

Customer (Public Portal)

1) Select a service, 2) answer a small set of qualifying questions, 3) provide preferred contact method and contact details, 4) submit the request, 5) receive a Request ID (confirmation).

Staff/Admin (Secret Dashboard)

1) View new requests, 2) open a request to see details and answers, 3) contact the customer externally, 4) record activities in a timeline (calls/WhatsApp/notes), 5) move the request through the pipeline, 6) review analytics to measure performance and conversion.

4. Product Scope (MVP Features)

Area	MVP Includes	MVP Excludes
Public Portal	Service listing; request form; validation; confidentiality pages with Request ID	User accounts; file uploads; chat
Admin Dashboard (Secret link)	Pipeline views; request details; activity timeline; status updates; loss reasons; analytics; automation	Reports; permissions; user management; notifications
Analytics	Funnel counts; conversion rates; average time to first contact	Advanced contact analysis; attribution; Breakouts; filters by

5. Pipeline & Status Rules

The pipeline is fixed in MVP to keep UX and analytics consistent:

- **New:** request submitted; no staff action recorded yet.
- **Contacted:** staff attempted or completed a contact (recorded activity).
- **Qualified:** staff confirmed the request is a potential client worth pursuing.
- **Won:** request converted to real business.
- **Lost:** request will not proceed (requires a reason).

6. Data Model (High Level)

Keep a single **requests** table with a **status** column rather than multiple tables per stage. This is a common production pattern and makes analytics and maintenance simpler.

Entity	Purpose	Key Fields (MVP)
tenants	Business/workspace boundary (multi-tenant readiness)	business, public_form_slug, admin_secret
services	Services offered by a tenant.	name, description
requests	Customer-submitted service requests/leads.	contact info, service_id, answers_json, status, lost_reason
activities	Timeline logs for follow-ups.	type (call/whatsapp/note), note, created_at

7. API Requirements (REST)

All endpoints are versioned and return JSON. The admin surface is protected by a secret in the URL for MVP. Validate inputs and return consistent error responses.

Public

```
GET /api/v1/tenants/{tenantSlug}/services
POST /api/v1/tenants/{tenantSlug}/requests
GET /api/v1/requests/{requestId} (optional: confirmation fetch)
```

Admin (secret link)

```
GET /api/v1/admin/{adminSecret}/requests?status=new&from=&to=&serviceId=
GET /api/v1/admin/{adminSecret}/requests/{requestId}
POST /api/v1/admin/{adminSecret}/requests/{requestId}/activities
PATCH /api/v1/admin/{adminSecret}/requests/{requestId}/status
GET /api/v1/admin/{adminSecret}/analytics?from=&to=&serviceId=
```

8. UI Requirements

Public UI

- Service cards/list with short descriptions.
- Short request form: 1–3 qualifying questions depending on service.
- Contact method selector (call / WhatsApp / email).
- Confirmation page showing Request ID.

Admin UI (Secret Dashboard)

- Pipeline view (tabs or kanban columns) for New / Contacted / Qualified / Won / Lost.
- Request details page including answers and contact info.
- Activity timeline list + add-activity form.
- Change status actions; Lost requires a reason.
- Analytics page with counters, rates, and time-to-first-contact.

Dark Mode

A toggle button in the top navigation switches Light/Dark themes. Preference is persisted in localStorage and applied on app load.

9. Non-Functional Requirements

- Stateless API (sessionless).
- Input validation and safe error handling.
- Reasonable performance for a small business workload (e.g., hundreds to low thousands of requests).
- Database migrations included (Prisma migrations).
- Containerized for local development using docker-compose.
- Logging: structured logs to stdout (JSON or key-value).
- Health endpoint for readiness/liveness (future K8s).

10. Tech Stack & Tooling (Free/Open Source)

Layer	Choice	Notes
Backend	Node.js + Express + TypeScript	Simple, familiar, fast iteration.
ORM/Migrations	Prisma	Schema-first, easy migrations.
Database	PostgreSQL	Relational + JSON support (answers_json).
Frontend	React + Vite + Tailwind CSS	Modern, fast, easy dark mode.
Local Dev	Docker + Docker Compose	API + DB locally reproducible.
Local K8s	k3d or kind	Free local Kubernetes cluster.
CI	GitHub Actions (free tier)	Build/test/lint; later image build/push.

11. Acceptance Criteria (MVP)

- A customer can submit a request for a selected service and receives a Request ID.
- Staff can open the secret admin dashboard, view requests by status, and open request details.
- Staff can add timeline activities and move requests through the fixed pipeline.
- Lost status requires selecting/entering a reason.
- Analytics page shows funnel counts and conversion rates over a date range.
- Dark mode toggle works and persists across reloads.
- Project runs locally with docker-compose using a single command.
- Repository includes a clear README with setup instructions.

Appendix A - AI Agent Build Prompt

You are a senior full-stack engineer building a production-like Mini SaaS called IntakeFlow.

Goal:

Build an MVP Service Intake & Lead Pipeline web application.

Constraints:

- Use only free and open-source tools (no paid services)
- Keep architecture simple and production-sane
- No authentication/authorization system in MVP (admin access via secret link)
- Single backend service + single database + single frontend
- Containerized with Docker Compose for local development
- Write clean TypeScript and modular code

Tech Stack (must use):

Backend: Node.js + Express + TypeScript + Prisma

Frontend: React + Vite + Tailwind CSS (with Dark Mode toggle)

Database: PostgreSQL

Product Behavior:

Customer:

- Select service
- Answer 1-3 qualifying questions
- Provide preferred contact method (call/WhatsApp/email) and contact details
- Submit request and receive a Request ID confirmation

Staff/Admin:

- Access dashboard via secret URL (no login)
- View requests grouped by fixed pipeline status: new, contacted, qualified, won, lost
- Open request details (including answers_json)
- Add timeline activities (call/whatsapp/note)
- Update request status; lost requires a reason
- View analytics counters: funnel counts, conversion rates, and average time-to-first-contact with date range

API (REST, JSON, versioned):

Public:

GET /api/v1/tenants/{tenantSlug}/services
POST /api/v1/tenants/{tenantSlug}/requests

Admin (secret in URL):

GET /api/v1/admin/{adminSecret}/requests?status=&from=&to=&serviceId=
GET /api/v1/admin/{adminSecret}/requests/{requestId}
POST /api/v1/admin/{adminSecret}/requests/{requestId}/activities
PATCH /api/v1/admin/{adminSecret}/requests/{requestId}/status
GET /api/v1/admin/{adminSecret}/analytics?from=&to=&serviceId=

Data Model (Prisma):

tenants: id, name, public_form_slug, admin_secret, created_at
services: id, tenant_id, name, description
requests: id, tenant_id, service_id, name, phone, email, preferred_contact_method, answers_json (JSON), status
activities: id, request_id, type, note, created_at

UI Requirements:

- Modern minimal SaaS style (cards, soft shadows, clean spacing)
- Dark mode toggle in top navbar; persist preference in localStorage
- Public page: service list + request form + confirmation page
- Admin page: pipeline tabs/columns + request detail + activity timeline + analytics page

Implementation Tasks (deliverables):

- 1) Scaffold monorepo: backend/ and frontend/
- 2) Prisma schema + migrations
- 3) Express API routes + validation + error handling + health endpoint
- 4) React UI pages + API client module
- 5) Dockerfiles + docker-compose.yml (api + db + frontend)
- 6) Seed script to create 1 tenant, 3 services, and an admin secret
- 7) README with commands to run locally

Do not over-engineer. Prefer clarity over abstraction. Provide runnable code.